

SEMANA 7 – APIS, GET, POST Y ALMACENAMIENTO EN MEMORIA

Actividad: Servidor con almacenamiento de reportes en array

<https://github.com/dannysj0214-commits/Programadores-para-la-paz/tree/3afcce4e0edd57e4feff75e453d3a6b197eaae3f/semana7>

1. Preguntas de selección múltiple

Pregunta 1:

¿Qué es un array en programación?

Respuesta: B. Una estructura que permite almacenar varios datos en una lista

Explicación:

Un array es una estructura de datos que permite almacenar múltiples valores en una sola variable. Los arrays son fundamentales en programación porque permiten agrupar elementos relacionados y acceder a ellos mediante índices. En JavaScript, los arrays pueden contener diferentes tipos de datos y tienen métodos integrados para manipularlos.

Pregunta 2:

¿Qué método HTTP se utiliza normalmente para consultar datos existentes?

Respuesta: B. GET

Explicación:

El método HTTP GET se utiliza para solicitar datos de un servidor. Es el método más común para consultar información y debe ser idempotente (no debe modificar el estado del servidor). En una API REST, GET se usa típicamente para recuperar recursos o listas de recursos.

Pregunta 3:

¿Qué método HTTP se usa normalmente para agregar información nueva?

Respuesta: B. POST

Explicación:

El método HTTP POST se utiliza para enviar datos al servidor y crear nuevos recursos. A diferencia de GET, POST modifica el estado del servidor y no es idempotente (cada solicitud puede tener un resultado diferente). En una API REST, POST se usa típicamente para crear nuevos registros o recursos.

Pregunta 4:

¿Cuál es el propósito de una API en una aplicación?

Respuesta: B. Permitir que diferentes sistemas o aplicaciones intercambien información

Explicación:

Una API (Application Programming Interface) es un conjunto de reglas y protocolos que permiten que diferentes aplicaciones o sistemas se comuniquen entre sí. Las APIs definen cómo se deben estructurar las solicitudes y respuestas, permitiendo que los desarrolladores integren servicios y compartan datos de manera estandarizada y segura.

2. Código del servidor (server.js)

Archivo: server.js

javascript

```
const express = require('express');
```

```
const app = express();
```

```
// Middleware para parsear JSON
```

```
app.use(express.json());
```

```
// Array para almacenar reportes en memoria
```

```
let reportes = [];
```

```
// Ruta GET /reportes - Consultar todos los reportes
```

```
app.get('/reportes', (req, res) => {
```

```
    res.json(reportes);
```

```
});
```

```
// Ruta POST /reportes - Registrar un nuevo reporte
```

```
app.post('/reportes', (req, res) => {
```

```
    const reporte = {
```

```
        id: reportes.length + 1,
```

```
        tipo: req.body.tipo,
```

```
        descripcion: req.body.descripcion
```

```
    };
```

```
reportes.push(reporte);
```

```
res.json({  
  mensaje: "Reporte registrado",  
  reporte: reporte  
});  
});
```

```
// El servidor escucha en el puerto 3000  
app.listen(3000, () => {  
  console.log('Servidor ejecutándose en puerto 3000');  
});
```

Ejecución del servidor:

```
bash  
node server.js
```

Rutas disponibles:

Método	Ruta	Descripción
GET	/reportes	Consultar todos los reportes registrados
POST	/reportes	Registrar un nuevo reporte

3. Prueba de reportes (prueba-reportes.txt)

Paso 1: Registrar un reporte (POST)

Petición POST a /reportes:

```
json  
{  
  "tipo": "Infraestructura",
```

```
"descripcion": "Daño en alumbrado público"
}
```

Respuesta del servidor:

```
json
{
  "mensaje": "Reporte registrado",
  "reporte": {
    "id": 1,
    "tipo": "Infraestructura",
    "descripcion": "Daño en alumbrado público"
  }
}
```

Paso 2: Consultar reportes (GET)

Petición GET a /reportes:

Respuesta del servidor:

```
json
[
  {
    "id": 1,
    "tipo": "Infraestructura",
    "descripcion": "Daño en alumbrado público"
  }
]
```

Paso 3: Registrar un segundo reporte (POST)

Petición POST a /reportes:

```
json
{
  "tipo": "Ambiente",
  "descripcion": "Acumulación de residuos en una zona comunitaria"
```

```
}
```

Respuesta del servidor:

json

```
{
```

```
  "mensaje": "Reporte registrado",
```

```
  "reporte": {
```

```
    "id": 2,
```

```
    "tipo": "Ambiente",
```

```
    "descripcion": "Acumulación de residuos en una zona comunitaria"
```

```
  }
```

```
}
```

Paso 4: Consultar nuevamente (GET)

Petición GET a /reportes:

Respuesta del servidor:

json

```
[
```

```
{
```

```
  "id": 1,
```

```
  "tipo": "Infraestructura",
```

```
  "descripcion": "Daño en alumbrado público"
```

```
},
```

```
{
```

```
  "id": 2,
```

```
  "tipo": "Ambiente",
```

```
  "descripcion": "Acumulación de residuos en una zona comunitaria"
```

```
}
```

```
]
```

Explicación del funcionamiento:

1. El servidor mantiene un array reportes en memoria que almacena todos los reportes registrados.
 2. Cada nuevo reporte recibe un id automático basado en la longitud actual del array (reportes.length + 1).
 3. La ruta GET /reportes devuelve el array completo con todos los reportes registrados hasta el momento.
 4. La ruta POST /reportes agrega un nuevo reporte al array y devuelve una confirmación con el reporte creado.
 5. **Importante:** Los datos se almacenan en memoria, por lo que al reiniciar el servidor, el array se vacía. Esto es normal en esta práctica, ya que aún no se utiliza una base de datos persistente.
-

4. Ejemplos de reportes ciudadanos (ejemplos-reportes.txt)

Ejemplo 1 - Infraestructura:

```
json
{
  "tipo": "Infraestructura",
  "descripcion": "Daño en alumbrado público"
}
```

Ejemplo 2 - Ambiente:

```
json
{
  "tipo": "Ambiente",
  "descripcion": "Acumulación de residuos en una zona comunitaria"
}
```

Ejemplo 3 - Seguridad:

```
json
{
  "tipo": "Seguridad",
  "descripcion": "Falta de señalización vial en la esquina de la calle principal"
}
```

Ejemplo 4 - Servicios públicos:

```
json
{
  "tipo": "Servicios públicos",
  "descripcion": "Baja presión de agua en el barrio Los Alamos durante las mañanas"
}
```

Ejemplo 5 - Movilidad:

```
json
{
  "tipo": "Movilidad",
  "descripcion": "Semáforo dañado en la intersección de la carrera 7 con calle 12"
}
```

5. Reflexión de la semana (reflexion-semana7.txt)

¿Por qué es importante que las comunidades puedan registrar y consultar reportes o incidencias en una plataforma tecnológica?

Las plataformas tecnológicas que permiten registrar y consultar reportes son herramientas esenciales para el empoderamiento ciudadano y la mejora de la calidad de vida en las comunidades. Cuando los ciudadanos tienen un canal accesible para reportar problemas, se sienten escuchados y parte activa de la solución, lo que fortalece el tejido social y la confianza en las instituciones.

El registro estructurado de incidencias permite a las autoridades y organizaciones comunitarias tener una visión clara y actualizada de los problemas que afectan a la población. Esto facilita la priorización de recursos y la planificación de acciones correctivas, evitando que los problemas se agraven por falta de información oportuna. Además, el historial de reportes permite identificar patrones y tendencias que podrían pasar desapercibidos en un sistema no estructurado.

La consulta de reportes también es fundamental porque brinda transparencia y rendición de cuentas. Los ciudadanos pueden verificar que sus reportes han sido recibidos y están siendo atendidos, lo que genera confianza en el sistema. También permite a la comunidad conocer qué problemas son más frecuentes y cómo se están abordando, fomentando una cultura de participación informada.

En el contexto de una plataforma comunitaria, la capacidad de registrar y consultar reportes convierte la tecnología en un puente entre la ciudadanía y las autoridades. No solo agiliza la comunicación, sino que también democratiza el acceso a la información y fortalece la capacidad de las comunidades para gestionar sus propios asuntos.

Desde una perspectiva técnica, el uso de GET para consultar y POST para registrar crea un flujo de trabajo intuitivo y estandarizado que facilita el desarrollo de aplicaciones y la integración con otros sistemas. La separación clara de responsabilidades entre las operaciones de lectura y escritura mejora la organización del código y la seguridad de la aplicación.

Además, el almacenamiento en memoria, aunque temporal, permite a los desarrolladores y a las comunidades probar y validar el funcionamiento del sistema antes de implementar soluciones más robustas como bases de datos. Esto es especialmente valioso en entornos comunitarios con recursos limitados.

En conclusión, las plataformas que permiten el registro y consulta de reportes son herramientas poderosas para la participación ciudadana, la transparencia y la mejora continua de los espacios comunitarios. Una comunidad que utiliza tecnología para gestionar sus necesidades se vuelve más organizada, participativa y capaz de resolver sus propios problemas de manera colaborativa.

Fecha de entrega: 26/06/2026